News Bias & Framing Detector

System Architecture, ML/NLP Pipeline & Technical Interview Guide

Executive Summary: News Bias Detector is a portfolio-grade, automated data pipeline and analytical dashboard that ingests news articles from 15+ global and Indian outlets, classifies political bias, sentiment, and entity portrayal using Google Gemini LLM, clusters articles reporting on the same real-world event using Sentence Embeddings & DBSCAN, and visualizes coverage gaps and narrative polarization on a live Streamlit dashboard.

1. System Architecture & Pipeline Breakdown

The project follows a decoupled, sequential ETL (Extract, Transform, Load) architecture divided into 4 modular layers:

Layer	Component & Technology	Function & Core Output
Layer 1: Ingestion	scraper.py feedparser · newspaper4k	Polls RSS feeds from 15+ outlets. Extracts clean article text, removes paywalled/short stubs (<100 chars), deduplicates by URL, enforces a 10-day rolling window. Output: articles.json.
Layer 2: Analysis	analyzer.py Google Gemini 3.8 Flash	Zero-shot LLM reasoning with strict JSON Schema. Extracts: **Bias Label** (Left→Right), **Sentiment** (-1.0 to +1.0), **Framing** (Alarmist, Neutral, etc.), and **Entity Portrayals**. Incremental processing saves API calls. Output: analyzed.json.
Layer 3: Clustering	clusterer.py all-MiniLM-L6-v2 · DBSCAN	Embeds Title + Body text into dense 384-d vectors. Uses cosine distance matrix + DBSCAN to identify cross-outlet coverage of identical events. Generates headline-overlap cluster labels. Output: clustered.json.
Layer 4: Dashboard	app.py Streamlit · Plotly · Pandas	Interactive visualization with 4 analytical views: **Coverage Gaps Matrix**, **Entity Sentiment Heatmap**, **Bias Radar**, and **Article Browser**. Non-blocking UI.

2. Automation & Cloud Infrastructure

A key design highlight is the **complete decoupling of heavy compute from web serving**:

- **Automated 3-Day ETL via GitHub Actions**: A scheduled cron workflow (0 0 */3 * *) runs in GitHub's cloud environment. It executes scraping, LLM classification (with rate-limit throttling of 14 RPM to respect Google's free tier), and event clustering. It automatically commits updated data files back to the repository.
- **Lightweight Web Hosting on Render**: Render hosts only the Streamlit visualization layer. Because Render free tier has a 512MB RAM cap, PyTorch and Sentence-Transformers are excluded from Render's build. Render builds in <20 seconds using a lean requirements.txt.

• **Continuous Deployment (CD)**: Whenever GitHub Actions commits fresh data to main, Render automatically detects the push and redeploys the dashboard without any manual intervention.

3. Technical Design Decisions: Why These Tools?

Why DBSCAN over K-Means for Event Clustering?

K-Means requires specifying *k* (the number of clusters) upfront, which is impossible in real-world news where the number of breaking stories fluctuates hourly. DBSCAN is density-based, discovers arbitrary numbers of clusters automatically, and marks isolated/unique stories as noise points (outliers) instead of forcing them into unrelated clusters.

Why Dense Sentence-Transformers over TF-IDF / BM25?

News outlets use radically different vocabulary to describe the same event (e.g., *'President signed executive decree'* vs *'White House enacts controversial policy order'*). TF-IDF fails because word overlap is zero. Sentence-Transformers (all-MiniLM-L6-v2) captures latent semantic meaning in 384-dimensional dense space.

Why Structured JSON Output with Gemini 3.8 Flash?

Gemini 3.8 Flash provides sub-second inference, low cost, and native JSON schema enforcement via response_json_schema. This eliminates common LLM hallucination and parsing issues, guaranteeing strict typing for downstream Pandas pivoting.

Why Incremental Processing & URL Hashing?

Analyzing 200+ articles from scratch on every run would burn API rate limits. analyzer.py loads existing analyzed.json, cross-references URLs, and only sends net-new articles to the LLM, preserving rate limits and execution speed.

4. Technical Interview Cheatsheet (Questions & Answers)

Here are the top questions an interviewer will ask about this project, along with optimal, high-scoring answers:

Q1: Walk me through the high-level architecture of your project.

Answer: 'The project is an automated end-to-end news intelligence system. It operates in 4 stages: First, a scraping engine collects RSS feeds from 15 global outlets using Feedparser and Newspaper4k. Second, an LLM pipeline uses Google Gemini 3.8 Flash with strict JSON schema to evaluate political bias, sentiment, framing, and entity portrayal. Third, an NLP clustering pipeline embeds headlines with MiniLM sentence transformers and clusters identical events using DBSCAN. Finally, an interactive Streamlit app visualizes media bias polarization, entity heatmaps, and coverage gaps. The entire pipeline is automated using GitHub Actions on a 24-hour (daily) cron and deployed on Render.'

Q2: How do you prevent the LLM from producing invalid or hallucinated data?

Answer: 'I used Google Gemini's native structured outputs feature with a strict JSON Schema definition. The schema enforces enum constraints on categorical fields like bias_label (Left, Center, Right), restricts sentiment_score to a float between -1.0 and 1.0, and bounds entity portrayals. Additionally, the code includes a fallback JSON parser with exponential retry logic to handle transient API hiccups.'

Q3: How did you handle API rate limits during batch processing?

Answer: 'Google Gemini's free tier has a 15 Requests-Per-Minute (RPM) limit. To avoid 429 Resource Exhausted errors, I implemented deterministic client-side pacing with a 4.2-second sleep interval (max ~14 RPM), coupled with exponential backoff on HTTP 429 responses. Furthermore, the pipeline is incremental: it checks already-analyzed URLs in our persistent cache and processes batches of 30 articles per run.'

Q4: Why did you pick DBSCAN for clustering and how did you tune it?

Answer: 'In news data, we do not know the number of breaking news stories in advance, which rules out K-Means. DBSCAN is ideal because it finds clusters based on spatial density and naturally isolates outlier articles as noise (-1). I precomputed cosine distances between L2-normalized sentence embeddings, set min_samples=2 so an event requires at least two outlets to form a cluster, and tuned eps=0.35–0.40 to balance cluster tightness with recall.'

Q5: How does the application maintain freshness in production?

Answer: 'We use an automated CI/CD schedule in GitHub Actions that triggers every 24 hours (daily). It executes the ETL pipeline, computes new clusters, and commits the updated JSON artifacts back to Git with [skip ci]. Render detects the repository commit and triggers a zero-downtime redeployment of the web UI. On the dashboard itself, we maintain a rolling 10-day retention window to prune stale news.'

Q6: How did you address cloud memory constraints on Render's free tier?

Answer: 'Render free tier has a 512MB RAM ceiling. Heavy ML libraries like PyTorch and HuggingFace Sentence-Transformers easily exceed 1GB during initialization. I decoupled compute from presentation: all model inference and clustering run on GitHub Actions runners (which provide 7GB RAM for free), while Render only installs lightweight visualization libraries (streamlit, plotly, pandas), reducing build time to under 20 seconds and memory to ~80MB.'

Q7: What is the Coverage Gap analysis and how is Polarization scored?

Answer: 'Coverage Gaps identify which outlets reported on a specific story and which outlets chose to ignore it—a key indicator of editorial agenda. For each event cluster, we calculate the standard deviation of sentiment across all reporting outlets as a **Polarization Score**. A high standard deviation highlights polarizing controversies where different media houses framed the exact same event in opposite emotional tones.'

5. System Scalability & Production Roadmap

If asked *'How would you take this to production at enterprise scale?'*, highlight these 4 architectural enhancements:

- **Vector Database Integration:** Replace in-memory DBSCAN distance matrix with a managed vector database (e.g. Qdrant, Pinecone, or Milvus with HNSW indexing) to scale from hundreds to millions of article embeddings with sub-millisecond approximate nearest neighbor searches.
- **Distributed Message Queue:** Decouple scraper workers from LLM workers using Celery / Redis or Apache Kafka to process articles asynchronously across distributed worker nodes.
- **Relational / Time-Series DB:** Migrate from flat JSON files to PostgreSQL + pgvector, enabling SQL queries on historical bias shifts, outlet trends over 12 months, and complex aggregation queries.
- **Multi-Modal & Video Bias:** Expand ingestion to YouTube news transcripts and podcast audio using Whisper for automated transcription prior to LLM analysis.